

Projet Python / VBA

Décisions de départ

IDE	VS Code
Langage coeur	Python orienté objet, typé et testé
Interface	Streamlit, strictement séparé du moteur de calcul
Versioning	Git + repository GitHub privé, travail par branches courtes
Environnement	Conda / Python 3.12, environnement reproductible
Principe d'architecture	Arbre de Node reliés directement entre eux ; aucune structure intermédiaire pour accéder aux noeuds
Priorité	Exactitude numérique, qualité du code, explicabilité, convergence et performance avant l'UI

Philosophie du projet. L'interface doit rendre le projet agréable à démontrer, mais elle ne doit jamais porter la logique quantitative. Le moteur Python doit pouvoir être testé et exécuté sans Streamlit. Cette séparation laisse la porte ouverte à une interface Excel/xlwings ultérieure sans réécrire le pricing.

1. Ce que le projet doit démontrer

Objectif central du cours : construire un **lattice binomial / trinomial** et un **option pricer**, puis utiliser ce projet pour démontrer une maîtrise conjointe de la finance quantitative, de la programmation orientée objet et de la performance numérique.

Socle demandé

	Attente	Statut
1	Construire un lattice binomial et trinomial, en commençant avec peu de pas, sans dividende et en exercice européen.	Obligatoire
2	Mettre autant que possible la logique dans des classes.	Obligatoire
3	Vérifier la convergence vers la formule de Black et étudier l'écart en fonction du nombre de pas et du strike.	Obligatoire
4	Ajouter l'exercice américain, étudier American – European, l'effet du signe du taux et les cas d'égalité.	Obligatoire
5	Ajouter le dividende discret et, pour $r = 0$, étudier son impact sur call et put.	Obligatoire
6	Implémenter les dérivées/Greeks demandées, notamment delta et gamma via les noeuds autour de la racine.	À intégrer

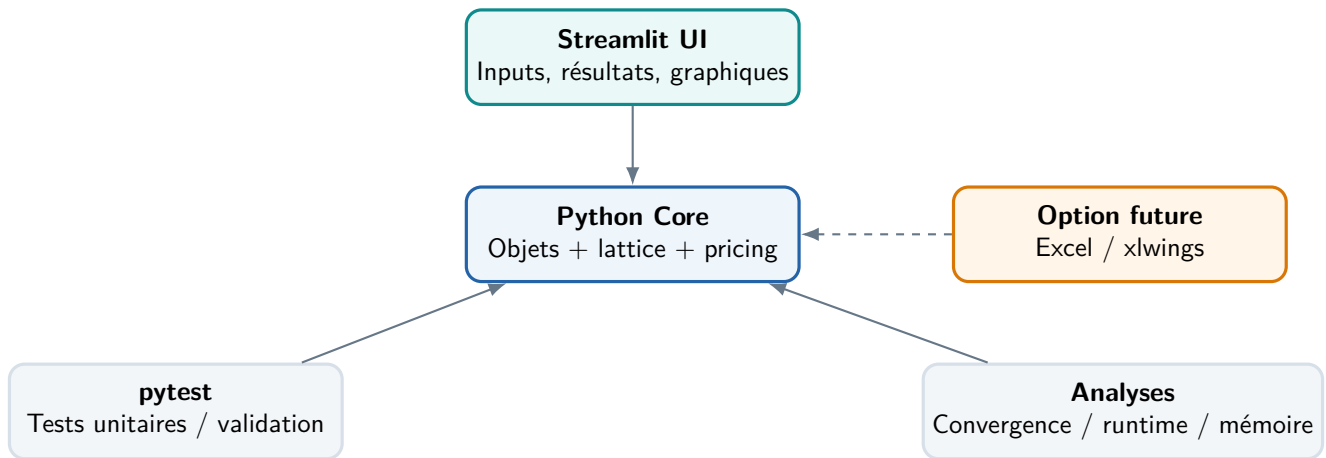
Extensions explicitement suggérées dans le cours

- visualisation de l'arbre : valeur de l'option et frontière d'exercice ;
- recherche de la limite du nombre de pas en temps et en mémoire ;
- pruning des noeuds de très faible probabilité et mesure de son effet sur 1000 pas ;
- choix automatique du nombre de pas à partir d'une précision définie par l'utilisateur.

Contrainte d'architecture majeure : pour accéder aux noeuds, le cours demande de ne pas utiliser d'objet intermédiaire : **pas de dictionnaire, pas d'array, pas de liste**. Les connexions doivent être portées directement par les noeuds eux-mêmes.

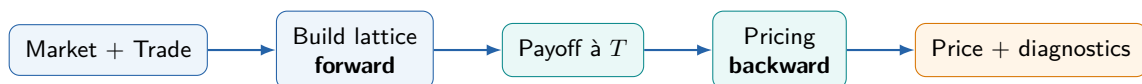
Hors périmètre initial : la partie "pyramid / 2-dimensional lattice" apparaît comme prolongement du cours, mais n'est pas listée dans les pages d'assignment. Nous la gardons comme extension éventuelle, uniquement si le socle est terminé, testé et propre.

2. Architecture cible : coeur indépendant de l'interface



Règle de conception : aucun calcul de prix, aucune probabilité de transition et aucune règle d'exercice ne doit vivre dans le code Streamlit. L'UI collecte des paramètres, instancie les objets du coeur et affiche les résultats.

Flux de calcul

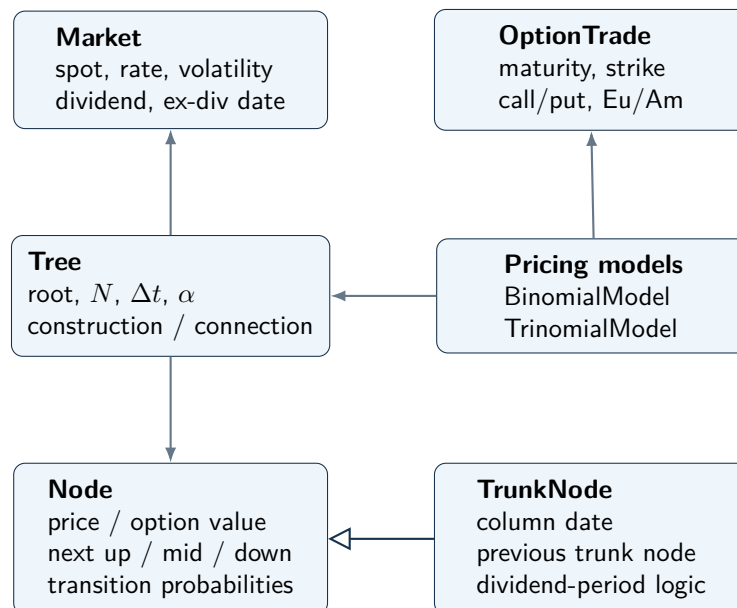


Pourquoi Streamlit reste un bon choix

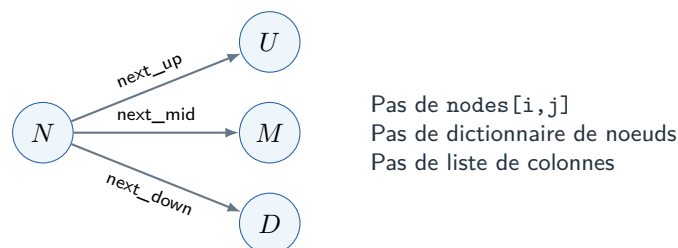
Atout	Conséquence pour le projet
Expérience du binôme	Très peu de temps perdu sur la plomberie d'interface.
Visualisation	Convergence, runtime, arbre et frontière d'exercice deviennent faciles à présenter.
Débogage	Le moteur s'exécute comme du Python normal, sans dépendance à Excel.
Évolutivité	Une UI Excel peut être ajoutée plus tard en appelant le même coeur.
Démonstration orale	Paramètres modifiables et résultats instantanément visibles.

3. Modèle objet proposé

Les noms ci-dessous sont volontairement proches de ceux du cours (Node, TrunkNode, Tree, OptionTrade, modèles binomial/trinomial). Le détail des attributs pourra évoluer pendant les séances : on ne fige que les responsabilités.



Structure de noeuds : liens directs



Attention aux “collections cachées”. Même si une liste ou un tableau serait pratique pour dessiner l'arbre, il ne doit pas devenir le mécanisme d'accès du moteur. Pour les visualisations, on pourra parcourir les liens et produire une structure temporaire d'affichage *hors du coeur*, uniquement si cela respecte l'interprétation attendue du professeur.

Principe financier à préserver

- le tree dépend du marché, du nombre de pas et de l'échéance ; strike, call/put et Eu/Am ne doivent pas piloter sa construction ;
- le binomial est le cas naturel sans dividende discret ; le cours bascule au trinomial lorsqu'un dividende discret doit être pris en compte ;
- l'exercice américain se traite au backward par comparaison entre valeur de continuation et valeur d'exercice.

4. Repository et organisation du code

```
lattice-option-pricer/
|-- README.md
|-- environment.yml
|-- .gitignore
|-- app/
| '-- streamlit_app.py
|-- src/
| '-- lattice_pricer/
|   |-- __init__.py
|   |-- market.py
|   |-- option.py
|   |-- node.py
|   |-- tree.py
|   |-- models.py
|   |-- pricer.py
|   |-- black_reference.py
|   '-- analysis.py
|-- tests/
|   |-- test_node.py
|   |-- test_tree.py
|   |-- test_pricing.py
|   '-- test_convergence.py
|-- docs/
|   |-- figures/
|   '-- report/
'-- notebooks/ # optionnel; jamais le coeur du projet
```

Structure minimale, pas sacrée. On ne créera un fichier que lorsqu'il contient une responsabilité claire. Si `pricer.py` ou `analysis.py` restent artificiels au début, ils n'existent pas encore. L'objectif est d'éviter l'over-engineering.

Conventions de code

Règle	Application
Fonctions courtes	Viser des méthodes courtes et mono-responsabilité; le CheckStyle du cours pénalise les méthodes longues.
Typage	Annoter paramètres et retours; typer les attributs importants.
Nommage	Classes en PascalCase; fonctions/variables en snake_case.
Visibilité	Préfixe <code>_</code> pour les membres internes quand pertinent.
Pas de globals	Les paramètres de marché et de modèle vivent dans les objets.
Commentaires	Expliquer la logique ou le choix numérique, pas paraphraser chaque ligne.
Défensif	Assertions / validations sur probabilités, dates, paramètres et incohérences de modèle.
Duplication	Une formule ou logique commune n'existe qu'à un seul endroit.

Workflow Git pour deux personnes



Branches types : `feature/binomial-tree`, `feature/american-exercise`, `analysis/convergence`, `fix/dividend-date`.

5. Environnement à installer

Stack retenue

Composant	Choix
OS	Windows 11
IDE	VS Code
Python	3.12 dans un environnement Conda dédié
Interface	Streamlit
Calcul	Python standard + NumPy uniquement lorsqu'il apporte une vraie valeur
Graphiques	Matplotlib au départ ; Plotly seulement si une visualisation interactive le justifie
Tests	pytest
Qualité	CheckStyle du professeur + Ruff comme garde-fou local
Versioning	Git + GitHub privé
Excel	Non requis dans le socle ; xlwings reste une extension possible

Dépendances initiales

```
python=3.12
streamlit
numpy
matplotlib
pytest
ruff
```

Pas de SciPy obligatoire. La référence Black–Scholes peut être écrite avec `math.erf` pour la CDF normale. Cela garde le projet léger et rend la formule entièrement explicable.

Extensions VS Code recommandées

- **Python** (Microsoft) ;
- **Pylance** ;
- support Git intégré de VS Code ;
- GitLens est optionnel, pas nécessaire au projet.

Commandes que l'on utilisera après installation

```
conda create -n dauphine-lattice python=3.12
conda activate dauphine-lattice
python -m pip install streamlit numpy matplotlib pytest ruff

# Lancer l'interface
streamlit run app/streamlit_app.py

# Lancer les tests
python -m pytest

# Controle local de style
ruff check .
```

6. Validation quantitative et robustesse

Tests de référence à intégrer dès le début

Contrôle	
✓	Option européenne sans dividende : comparer le prix du tree à Black–Scholes et vérifier la convergence quand N augmente.
✓	Vérifier que les probabilités sont dans $[0, 1]$ et que leur somme vaut 1 à la précision numérique près.
✓	Call européen de strike 0 : comparer au forward à maturité, y compris avec dividende, comme suggéré dans le cours.
✓	Option américaine : contrôler la logique d'exercice et inspecter la frontière d'exercice.
✓	Dates de dividendes : traiter explicitement les égalités à une tolérance pour éviter les erreurs dues aux pas de temps fractionnaires.
✓	Cas aléatoires : générer des jeux de paramètres et enregistrer automatiquement les cas qui violent une assertion ou présentent un écart anormal.

Convergence et performance à mesurer



Le cours demande explicitement de chercher la vitesse de convergence du gap et la croissance du temps de calcul. Ces mesures doivent être produites par le coeur Python, puis uniquement visualisées par Streamlit.

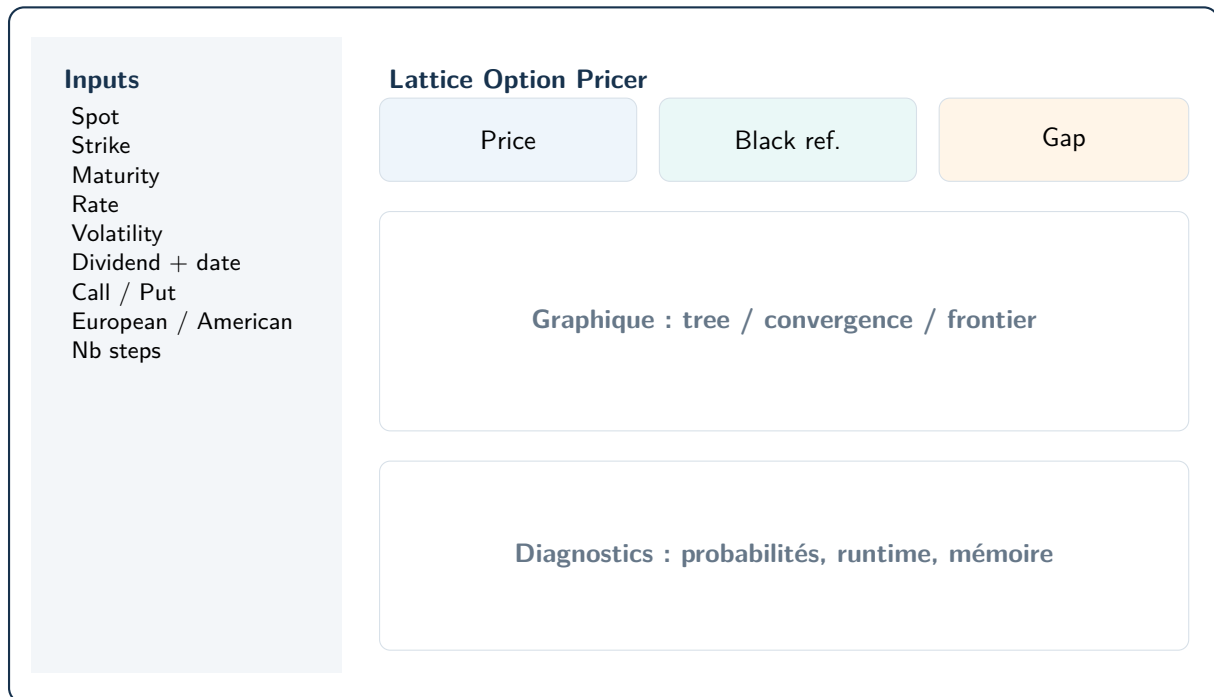
Précision numérique

Deux difficultés du cours sont à intégrer à notre démarche : **(1)** les dates proches peuvent tomber dans des périodes différentes à cause des erreurs numériques ; **(2)** certaines formules de probabilités peuvent devenir instables lorsque deux grandeurs très proches sont soustraites. On préférera les formulations algébriques simplifiées lorsqu'elles sont disponibles et on documentera le pourquoi.

Gestion mémoire

- ne pas conserver inutilement des colonnes du tree une fois pricées ;
- couper les connexions devenues inutiles afin que le garbage collector puisse libérer les objets ;
- mesurer l'effet avant/après, au lieu de supposer qu'une optimisation est utile ;
- éviter d'instrumenter en permanence `__new__`/`__del__` si cela dégrade fortement le runtime.

7. Interface Streamlit : sobre, utile et démontrable



Organisation recommandée de l'UI

- **Sidebar** : paramètres de marché, trade, modèle et discrétisation ;
- **Pricing** : prix, benchmark Black quand applicable, gap et temps de calcul ;
- **Tree** : arbre uniquement pour un nombre de pas faible afin de rester lisible ;
- **Convergence** : prix et gap vs. N ;
- **Exercise** : frontière d'exercice pour les options américaines ;
- **Performance** : runtime, nombre de noeuds / mémoire et comparaison pruning.

L'UI n'est pas un concours de design. Une fonction Streamlit ne doit pas masquer un calcul difficile à expliquer. Les graphes sont là pour rendre visibles les propriétés du modèle et les résultats demandés dans le report.

8. Roadmap de développement

Phase	Livrable	Critère de sortie
0	Environnement + repo	Même environnement sur les deux PC, tests exécutables, Streamlit démarre.
1	Objets de base	Market, OptionTrade, Node / TrunkNode testés ; liens directs définis.
2	Binomial simple	Quelques pas, aucun dividende, européen ; valeurs du tree contrôlables manuellement.
3	Trinomial simple	Recombinaison et probabilités valides ; même cas européen sans dividende.
4	Pricing + Black	Backward pricing, benchmark Black, premiers graphes de convergence.
5	American	Hold vs. exercise, étude American – European et frontière.
6	Dividendes	Bascule trinomial, traitement robuste de la date ex-dividend.
7	Greeks	Delta / gamma, fonctions de dérivation demandées.
8	Performance	Runtime, mémoire, pruning et montée en nombre de pas.
9	UI finale + report	Démo reproductible, beaux graphes, report court et centré sur résultats/difficultés.

Répartition de travail recommandée

Chaque feature doit avoir un **owner**, mais aucune brique critique ne doit être connue d'une seule personne. Pour les parties difficiles (construction/recombinaison du tree, probabilités, dividendes, memory), le binôme relit ensemble avant merge.

Definition of Done

Une feature est terminée seulement si : **(1)** elle fonctionne, **(2)** un test la couvre, **(3)** le code est court et compréhensible, **(4)** aucune logique métier n'est dupliquée, **(5)** les deux membres peuvent l'expliquer, **(6)** le résultat pertinent est mesuré ou visualisable.

Ordre de priorité si le temps devient contraint : exactitude et tests > architecture claire > exigences obligatoires > analyses de convergence/performance > extensions > sophistication graphique.

Prochaine étape : installation guidée de l'environnement sur Windows / VS Code, puis création du repository GitHub et du squelette minimal.